PriX: Private Accelerated Transformers via Secure Multi-Party Computation in JAX

Mehmet Ugurbil
mugurbil@gmail.com
https://github.com/mugurbil/prix

Abstract

Private inference of Transformer and Large Language Models (LLMs) via Secure Multi-Party Computation (MPC) faces a systems bottleneck: CrypTen-family frameworks execute secret-shared protocols over $\mathbb{Z}_{2^{64}}$ via eager, operator-by-operator dispatch in PyTorch. While recent algorithmic advances—Wavelet-encoded Look-Up Tables (CuRL), piecewise low-degree polynomials (PUMA, Nimbus), 1-round Fourier-series activations and ODE Softmax (SHAFT), and hybrid plaintext evaluators (Fission)—substantially reduce the cost of nonlinearities, their reference implementations are spread across separate codebases and pay kernel-launch and host-side dispatch overhead on every elementary ring operation.

We present **PriX**, an open-source JAX/XLA framework for private Transformer inference that implements five MPC approximation paradigms (**CrypTen**, **PUMA/Nimbus piecewise polynomials**, **SHAFT**, **CuRL**, and **Fission**) under a single API while compiling local ring operations into fused XLA kernels. PriX combines (1) *exact limb-decomposed FP64 matrix multiplication* over $\mathbb{Z}_{2^{64}}$ that computes only the limb products surviving modulo 2^{64} (6 products with 22-bit limbs for contraction size $K \leq 512$, 10 products with 16-bit limbs otherwise); (2) `@jax.jit` *fusion* of the local steps of multi-round MPC circuits (carry-lookahead comparison rounds, wavelet LUT gathers, Fourier-series combinations); and (3) *a shared-memory collective* that exchanges batched openings in a single buffer. Across 10 MPC primitives and 5 Transformer blocks (`BertTiny`, `BertBase`, `GPT2`, `Llama`, `ModernBert`), PriX has lower latency than the PyTorch CrypTen baseline in every configuration measured: all 75 on a CPU (five modes) and all 60 on an NVIDIA Tesla T4 GPU (four modes). Transformer-block speedups are **2.83**×–**9.20**× on CPU and **1.97**×–**3.09**× on GPU. Both frameworks run on a single host, so these measurements reflect computation and inter-process overhead rather than wide-area network cost.

1 Introduction

Deploying Transformer-based Large Language Models (LLMs) [16] on sensitive user prompts or proprietary model weights requires cryptographic privacy guarantees. Secure Multi-Party Computation (MPC) over the machine ring $\mathbb{Z}_{2^{64}}$ [11, 12, 8, 6] enables two or more distrusting parties to jointly evaluate neural networks on secret-shared inputs without revealing either the client's prompt or the server's parameters.

Over the past five years, CrypTen [8] has served as the foundational Python research platform for machine-learning-oriented MPC. Building upon CrypTen, a wave of recent cryptographic protocol innovations has dramatically improved the accuracy and round complexity of Transformer nonlinearities (GeLU, SiLU, Softmax, LayerNorm, RMSNorm):

- **Wavelet Approximations and CuRL** [5, 13, 14]: Wavelet-based cryptographic function approximation—pioneered across Fully Homomorphic Encryption in **Ripple** [5], mixed-mode MPC in **Wave Hello to Privacy** [13], and private LLM evaluation in **CuRL** [14]—compresses lookup tables via Haar and Biorthogonal-2.2 wavelet transforms paired with probabilistic truncation (TruncPr) [4], substantially reducing LUT communication and memory footprints while preserving high numerical fidelity.

- **SHAFT** [7] introduced 1-round trigonometric-triple Fourier-series evaluation for GeLU, SiLU, Erf, Sigmoid, and Tanh, alongside a constant-round Forward-Euler Ordinary Differential Equation (ODE) solver for Softmax.

- **Fission** [15] introduced a hybrid permute-and-reveal architecture that offloads elementwise nonlinearities to a non-colluding plaintext evaluator party.

- **PUMA** [3] and **Nimbus** [10] replace GeLU, SiLU, and exp with piecewise low-degree polynomials guarded by one or two secure comparisons, avoiding lookup tables entirely; Nimbus additionally fits the pieces to the observed input distributions of Transformer activations.

Despite these algorithmic advances, MPC research for Transformers faces two systems obstacles:

1. **Ecosystem Fragmentation:** The original CrypTen repository is no longer actively developed, and CuRL, SHAFT, and Fission live in separate CrypTen forks (PUMA lives in SecretFlow SPU), each of which needs compatibility patches to run on current Python, PyTorch, and JAX releases. Researchers conducting ablations across approximation paradigms must port models between distinct codebases.

2. **Eager Dispatch Overhead:** In eager PyTorch, every elementary bitwise shift, mask, and ring addition in an MPC circuit is a separate CPU or CUDA kernel launch with host-side Python control flow in between. A single 64-bit carry-lookahead comparison [9] executes 6 tree rounds of bitwise XOR and AND operations, so its local work is split into many tiny launches. In addition, because neither CPUs' BLAS libraries nor CUDA provide 64-bit integer (`int64`) GEMM, CrypTen's CUDA backend splits each operand into 3 (for contraction size $K < 256$) or 4 limbs and computes all 9 or 16 pairwise limb products in one batched FP64 `torch.matmul`, discarding the products that vanish modulo 2^{64} only afterwards; on CPU it falls back to PyTorch's unblocked `int64` matmul.

To address both challenges, we design and implement **PriX** (*Private Accelerated Transformers in JAX*), a modern, drop-in alternative to CrypTen built on the JAX/XLA compiler stack [2].

Contributions. Our contributions are:

1. **A Multi-Paradigm MPC Framework:** PriX implements **CrypTen** polynomial/Newton–Raphson approximations [8], **PUMA / Nimbus**-style piecewise low-degree polynomials [3, 10], **SHAFT** Fourier-series and ODE Softmax [7], **CuRL** wavelet LUTs with TruncPr [14, 4], and **Fission** hybrid evaluators [15] behind a single API on Python 3.10–3.13, tested on CPUs and CUDA GPUs. Researchers switch paradigms in one line (`mode="lut" | "poly" | "shaft" | "crypten" | "fission"`) or mix methods per function (e.g., SHAFT ODE Softmax with CuRL wavelet LayerNorm).

2. **XLA Kernel Fusion for MPC Circuits:** PriX compiles the local (non-communicating) steps of secret-share arithmetic, carry-lookahead comparison rounds, wavelet LUT gathers,

and Fourier-series combinations via `@jax.jit`, removing intermediate memory traffic and Python dispatch between elementary ring operations.

3. **Exact Limb-Decomposed $\mathbb{Z}_{2^{64}}$ Matrix Multiplication and Low-Overhead Collectives:** PriX computes only the limb products that survive modulo 2^{64} (6 instead of CrypTen's 9 for $K < 256$, 6 instead of 16 for $256 \le K \le 512$, and 10 instead of 16 for $K > 512$) inside one compiled kernel, and exchanges batched openings through a shared-memory ring buffer.

4. **CPU and GPU Evaluation:** We compare PriX against the PyTorch CrypTen baseline on 10 primitives and 5 Transformer blocks (`BertTiny`, `BertBase`, `GPT2`, `Llama`, `ModernBert`) on a CPU (all 5 modes, 75 configurations) and an NVIDIA Tesla T4 GPU (4 modes, 60 configurations). PriX has lower latency in every configuration, with **2.83×–9.20×** Transformer-block speedups on CPU and **1.97×–3.09×** on GPU. We also document the limits of these measurements: a single host, different local transports, and few repetitions.

2 Background and MPC Preliminaries

2.1 Fixed-Point Encoding and Secret Sharing over $\mathbb{Z}_{2^{64}}$

Real-valued tensors $x \in \mathbb{R}^{d_1 \times \cdots \times d_k}$ are encoded into the ring $\mathbb{Z}_{2^{64}}$ using a fixed-point scaling factor $S = 2^f$ (where $f = 16$ precision bits by default):

$$\mathsf{Encode}(x) = \lfloor x \cdot 2^f \rceil \bmod 2^{64}, \qquad \mathsf{Decode}(\hat{x}) = \frac{\mathsf{Signed}_{64}(\hat{x})}{2^f}. \tag{1}$$

Across N parties (typically $N = 2$), PriX maintains two secret-sharing representations:

- **Additive Arithmetic Sharing ($\langle x \rangle^A$):** Party $p \in \{0, \ldots, N-1\}$ holds $\langle x \rangle_p^A \in \mathbb{Z}_{2^{64}}$ such that $\sum_{p=0}^{N-1} \langle x \rangle_p^A \equiv \mathsf{Encode}(x) \pmod{2^{64}}$. Linear combinations $\langle x + y \rangle^A = \langle x \rangle^A + \langle y \rangle^A$ and public multiplications $\langle c \cdot x \rangle^A = c \cdot \langle x \rangle^A$ require zero communication (followed by local or interactive fixed-point truncation).

- **XOR Binary Sharing ($\langle x \rangle^B$):** Party p holds $\langle x \rangle_p^B \in \{0,1\}^{64}$ such that $\bigoplus_{p=0}^{N-1} \langle x \rangle_p^B = x$. Bitwise XOR ($\oplus$) and bit shifts ($\ll, \gg$) are local; bitwise AND ($\wedge$) requires 1 communication round via binary Beaver triples.

2.2 Multiplication, Truncation, and Nonlinearity Paradigms

Beaver Multiplication Triples. To compute the private product $\langle z \rangle^A = \langle x \cdot y \rangle^A$ (or matrix product $\langle XY \rangle^A$), the parties consume a pre-generated triple $(\langle a \rangle^A, \langle b \rangle^A, \langle c \rangle^A)$ with $c = a \cdot b \bmod 2^{64}$ [1]. Parties locally compute $\langle \epsilon \rangle^A = \langle x \rangle^A - \langle a \rangle^A$ and $\langle \delta \rangle^A = \langle y \rangle^A - \langle b \rangle^A$, open (ϵ, δ) in a single communication round, and evaluate:

$$\langle x \cdot y \rangle_p^A = \langle c \rangle_p^A + \epsilon \langle b \rangle_p^A + \langle a \rangle_p^A \delta + \mathbb{I}[p = 0] \cdot (\epsilon\delta) \pmod{2^{64}}. \tag{2}$$

Because multiplying two 2^f-scaled integers yields scale 2^{2f}, the product must be truncated by f bits:

- *CrypTen Wrap-Count Truncation (`crypten`):* Parties locally arithmetic-shift their shares by f bits and correct for unsigned ring wraps θ_x via a single-round masked wrap-count protocol [8].

- *Probabilistic Truncation (`egk`):* Following Escudero et al. [4] and CuRL [14], parties mask $\langle z \rangle^A$ with a correlated random pair $(\langle r \rangle^A, \langle r \gg f \rangle^A)$, reveal $z + r$, and subtract $\langle r \gg f \rangle^A$ from $(z + r) \gg f$.

Table 1: Summary of the five MPC approximation paradigms implemented in PriX (default configuration of each mode).

Mode	Truncation	GeLU / SiLU	Softmax	Reciprocal / InvSqrt
`"crypten"` [8]	Wrap-count	Taylor erf / Sigmoid	Secure max + limit exp + NR recip.	Newton–Raphson
`"poly"` [3, 10]	Wrap-count	Piecewise degree-4 poly	Secure max + piecewise exp + NR recip.	Newton–Raphson
`"shaft"` [7]	Wrap-count	1-round Fourier series	Forward-Euler ODE	Newton–Raphson
`"lut"` [14]	TruncPr [4]	Bior-2.2 LUT	Secure max + Bior-2.2 exp / recip. LUTs	Bior-2.2 LUT
`"fission"` [15]	Evaluator	Permute & reveal	Permute & reveal	Permute & reveal

Five Nonlinear Approximation Backends. Table 1 summarizes the five approximation modes unified in PriX:

1. **CrypTen Mode (`mode="crypten"`)** [8]: Approximates $\exp(x) \approx (1 + x/2^m)^{2^m}$ via $m = 8$ repeated squarings, $1/x$ and $1/\sqrt{x}$ via Newton–Raphson iterations, and $\mathrm{GeLU}(x)$ via Taylor-series $\mathrm{erf}(x/\sqrt{2})$.

2. **PUMA / Nimbus Piecewise Polynomial Mode (`mode="poly"`)** [3, 10]: Evaluates GeLU, SiLU, Tanh, Sigmoid, and Erf with one batched secure comparison (computing $[\langle x\rangle^A < 0]$ and $[|\langle x\rangle^A| < w]$ in a single carry-lookahead circuit) followed by a degree-4 polynomial on $|x|$, and exp with a two-piece degree-4 polynomial on $(-5, 0]$ and $[0, 2.5]$ whose pieces share one power basis. No lookup tables are used. The coefficients are least-squares fits over the stated intervals rather than the distribution-aware fits of Nimbus.

3. **SHAFT Mode (`mode="shaft"`)** [7]: Evaluates GeLU, SiLU, Erf, Sigmoid, and Tanh via a Fourier sine series $\sum_{k=1}^{K} \beta_k \sin(k\omega x)$ in *1 communication round*: parties reveal $\delta = x - t$ against a pre-shared trigonometric triple $(\langle t\rangle^A, \langle \sin(k\omega t)\rangle^A, \langle \cos(k\omega t)\rangle^A)$ and apply the angle-addition identity locally:

$$\langle \sin(k\omega x)\rangle^A = \sin(k\omega\delta)\langle \cos(k\omega t)\rangle^A + \cos(k\omega\delta)\langle \sin(k\omega t)\rangle^A. \tag{3}$$

   For Softmax, SHAFT integrates the differential equation $y'(t) = y(t) \odot (x - \langle y(t), x\rangle)$ over $t \in [0, 1]$ using a fixed number of Forward-Euler steps, without any division, reciprocal, or secure max.

4. **CuRL Wavelet LUT Mode (`mode="lut"`)** [14]: Builds on wavelet-based function approximation [5, 13]: lookup tables are compressed with Haar and Biorthogonal-2.2 wavelet transforms into coarse coefficient tables, secret-shared lookups are evaluated with one-hot correlated randomness [14], and Biorthogonal-2.2 tables are interpolated piecewise-linearly after probabilistic truncation TruncPr [4].

5. **Fission Evaluator Mode (`mode="fission"`)** [15]: Masks and randomly permutes shares before revealing them to a non-colluding plaintext evaluator party, which computes exact float32 nonlinearities and returns fresh secret shares. The evaluator learns the permuted plaintext values, so the security argument relies on the evaluator not colluding with the MPC parties and on partitioning values across evaluators [15].

3 PriX Architecture and XLA Compiler Optimizations

Figure 1 presents the four-layer architecture of PriX. Eager PyTorch frameworks launch each ring-arithmetic or bitwise operation as its own kernel. PriX instead expresses the local (non-communicating) phase between two openings of a protocol as a pure function over 64-bit JAX

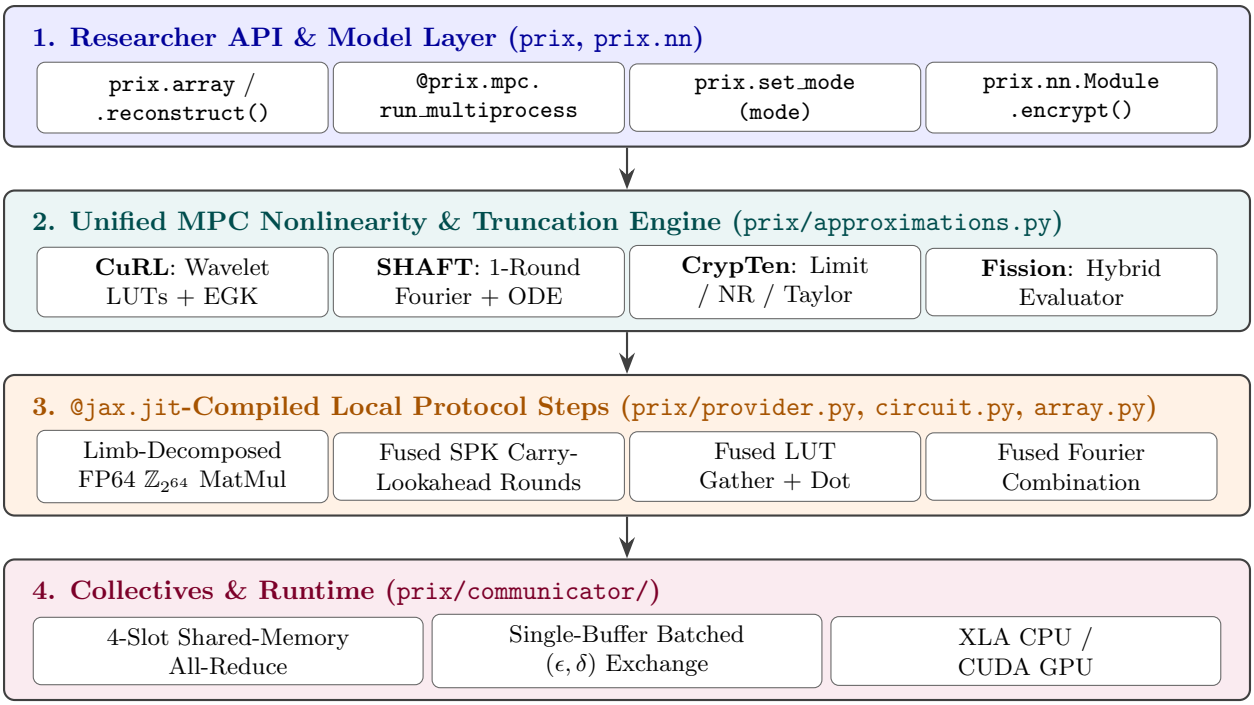


Figure 1: System architecture of **PriX**, from the researcher-facing Python API down to compiled local protocol steps and shared-memory collectives.

arrays (`jnp.int64` / `jnp.uint64`) and compiles it with `@jax.jit`, so each such phase becomes one XLA executable. Communication remains on the host between compiled phases.

3.1 Exact Limb-Decomposed FP64 Matrix Multiplication over $\mathbb{Z}_{2^{64}}$

Neither CPU BLAS libraries nor cuBLAS provide 64-bit integer (`int64`) GEMM, so PyTorch's `int64` matmul runs on generic unblocked loops. Double-precision GEMM, by contrast, is available in optimized form on both (AVX/FMA on CPU, `cublasDgemm` on GPU). Note that on the Tesla T4 used in our GPU experiments, FP64 throughput is a small fraction of FP32 throughput and does not use tensor cores.

To compute $C = AB \bmod 2^{64}$ for $A \in \mathbb{Z}_{2^{64}}^{M\times K}$ and $B \in \mathbb{Z}_{2^{64}}^{K\times N}$ exactly in FP64, we split each unsigned 64-bit entry into L limbs of w bits, $A = \sum_i A_i 2^{wi}$ and $B = \sum_j B_j 2^{wj}$. Because only the result modulo 2^{64} is needed, every limb pair with $(i+j)\,w \geq 64$ contributes a multiple of 2^{64} and can be skipped:

$$AB \equiv \sum_{(i+j)\,w<64} (A_i B_j)\, 2^{w(i+j)} \pmod{2^{64}}. \tag{4}$$

PriX uses two limb configurations, chosen by the contraction size K:

- $K \leq 512$: $L = 3$ limbs of $w = 22$ bits (A_2, B_2 have 20 bits), so only the **6** pairs with $i + j \leq 2$ are computed. Each product entry is below $K \cdot 2^{44} \leq 2^{53}$, so every FP64 GEMM is exact.
- $K > 512$: $L = 4$ limbs of $w = 16$ bits, so only the **10** pairs with $i + j \leq 3$ are computed. On GPU each pair is a separate slice of one batched GEMM. Each pair product is converted to `uint64` individually, which is exact for $K \cdot 2^{32} \leq 2^{53}$, i.e. $K \leq 2^{21}$. On CPU, each pair

product is likewise converted to `uint64` before products of equal shift are added, so both backends share the same $K \leq 2^{21}$ bound.

- $K > 2^{21}$: the contraction is split into chunks of at most 2^{21}. Each chunk is multiplied exactly as above, and the `uint64` partial results are added; wrap-around addition is exact modulo 2^{64}, so the result is bit-exact for any K. The chunking is decided at trace time, so it adds no overhead when $K \leq 2^{21}$.

CrypTen's CUDA backend [8] uses 3 limbs for $K < 256$ and 4 limbs otherwise, and computes *all* 9 or 16 limb products in one batched `torch.matmul` before discarding the vanishing ones. PriX therefore performs 6 instead of 9 GEMMs for $K < 256$, 6 instead of 16 for $256 \leq K \leq 512$, and 10 instead of 16 for $K > 512$. On GPU, PriX stacks the surviving limb pairs into one batched GEMM, and the limb extraction, GEMM, conversion, shifts, and modular accumulation are compiled together with the Beaver combination $\langle C\rangle^A + \epsilon\langle B\rangle^A + \langle A\rangle^A\delta + \mathbb{I}[p=0](\epsilon\delta)$. On CPU, CrypTen uses PyTorch's `int64` matmul directly, so PriX's advantage there comes mainly from using FP64 BLAS.

3.2 Compiled Set-Propagate-Kill Comparison Circuits

Private comparisons ($\langle x\rangle^A < 0$, ReLU, Max) extract the most significant bit of a secret-shared ring element by converting $\langle x\rangle^A$ into binary shares with a parallel-prefix carry-lookahead circuit. Like CrypTen, PriX uses the Set-Propagate-Kill (SPK) tree of Damgård et al., a Kogge–Stone-style prefix circuit [9] with $\log_2 64 = 6$ rounds. In round r with stride $s = 2^r$, the set and propagate bit-vectors are updated as

$$S^{(r+1)} = S^{(r)} \oplus \big(P^{(r)} \wedge (S^{(r)} \ll s)\big), \qquad P^{(r+1)} = P^{(r)} \wedge (P^{(r)} \ll s), \tag{5}$$

where XOR can replace OR because the two terms are never both set. As in CrypTen, both AND gates of a round are opened in one batched binary-Beaver exchange. PriX differs in how the local work is executed:

- The dealer generates the binary triples for all 6 rounds in one call (`generate_spk_triples`) instead of one call per round.
- The local work between two consecutive openings—applying the previous round's Beaver correction, masking, shifting, and preparing the next round's masked operands—is one compiled kernel (`_spk_step_fused` in `circuit.py`). The first and last rounds are fused with the arithmetic-to-binary conversion and with the binary-to-arithmetic conversion of the sign bit, respectively (`_a2b_post_and_spk_round0_pre`, `_spk_round_last_and_ltz_b2a_pre`, `_b2a_combine`).

3.3 Compiled Wavelet LUT Evaluation and Fourier Combination

In CuRL mode (`mode="lut"`), a nonlinearity such as InvSqrt, Exp, GeLU, or SiLU truncates the secret input to a table index, reveals the index masked by a one-hot correlation, rotates the one-hot share accordingly, and takes its dot product with the (Biorthogonal-2.2) coefficient tables before interpolating. In PriX:

- All wavelet tables are built once and placed on the accelerator at initialization (`LookupTables(device)`).

- The rotation and dot product against one table, or against the two Biorthogonal-2.2 tables used for interpolation, are one compiled kernel (`_lut_gather_dot`, `_bior_lut_gather_dot` in `array.py`).
- In SHAFT mode, after the single opening of $\delta = x - t$, the evaluation of Equation 3 for all K terms—computing $\sin(k\omega\delta)$ and $\cos(k\omega\delta)$, scaling them by β_k, multiplying against the triple shares, and summing—is one compiled kernel (`_fourier_linear_comb_jit` in `approximations.py`).

3.4 Batched Openings and Shared-Memory Collectives

Every private multiplication opens two masked tensors, $\epsilon = x - a$ and $\delta = y - b$. As in CrypTen, both are opened in one round. When ϵ and δ have different shapes (e.g. $\epsilon \in \mathbb{Z}_{2^{64}}^{M\times K}$ and $\delta \in \mathbb{Z}_{2^{64}}^{K\times N}$ in a matrix multiplication), PriX flattens and concatenates them on the device in a compiled kernel, copies the result to the host as a single buffer, exchanges it, and splits the result on the device.

For multi-process execution on one host, `InProcessCommunicator` uses a **4-slot shared-memory ring buffer** (`multiprocessing.RawArray`) with a 1-byte pipe message as the readiness signal. Each party writes its buffer into slot step mod 4, sends the slot index over the pipe, and adds its peer's slot directly from shared memory, without pickling tensors through queues. CrypTen instead uses `torch.distributed` with the `gloo` backend over loopback TCP. This difference affects every opening, so part of the end-to-end speedups in Section 5 is due to the communicator rather than to kernel fusion; we do not separate the two effects in this report.

4 Researcher Ergonomics and Unified API

A central design goal of PriX is enabling cryptography and machine learning researchers to prototype and benchmark private Transformer protocols with minimal boilerplate. Listing 1 shows a multi-party ablation that evaluates a pre-norm Transformer block under all five approximation modes.

Listing 1: Evaluating a private Transformer block under all five MPC approximation modes in PriX.

```
import numpy as np
import prix
import prix.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, d_model=64, n_heads=4, d_ff=256):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(
            d_ff, d_model))

    def forward(self, x):
        h = self.ln1(x)
        x = x + self.attn(h, h, h)
        return x + self.ffn(self.ln2(x))

def evaluate_private_block(tokens):
```

```
    # Party 0 secret-shares the model weights; party 1 secret-shares the token
        embeddings
    model = TransformerBlock().encrypt(src=0)
    x = prix.array(tokens, src=1)
    return model(x).reconstruct()

tokens = np.random.randn(1, 16, 64).astype(np.float32)
for mode in ["crypten", "poly", "shaft", "lut", "fission"]:
    out = prix.run(evaluate_private_block, tokens, world_size=2, mode=mode)
```

5 Experimental Evaluation

5.1 Experimental Setup

We compare **PriX (JAX)** against a PyTorch **CrypTen** baseline [8]: the CrypTen fork that serves as the reference implementation of CuRL [14] and Fission [15]. SHAFT's Fourier activations and ODE Softmax [7], and the piecewise polynomials of `poly` mode, are added to this baseline by our benchmark script, which ports PriX's algorithms to it one-to-one (same coefficients, intervals, comparisons, and power-basis sharing). Where PriX seeds Newton–Raphson iterations with the limit approximation of exp regardless of the configured exp method, the baseline is configured the same way. Both frameworks run 2-party semi-honest protocols with the Trusted First Party (`TFP`) dealer, $f = 16$ fractional bits, identical inputs, and identical per-mode configuration files (each mode's YAML merged over the framework's default configuration; the two frameworks' default configurations agree on every parameter used). The five modes are:

1. **CrypTen approximations** (`crypten`) [8];
2. **Piecewise polynomials** (`poly`) [3, 10];
3. **SHAFT Fourier + ODE** (`shaft`) [7];
4. **CuRL wavelet LUTs with** TruncPr (`lut`) [14]; and
5. **Fission hybrid evaluator** (`fission`) [15], with one evaluator process.

We benchmark 10 MPC primitives and nonlinearities (MatMul 128×128, Mul 128×128, ReLU 32×64, Exp 32×64, Reciprocal 32×64, InvSqrt 32×64, GeLU 32×64, SiLU 32×64, Softmax $4\times32\times32$, LayerNorm $4\times32\times64$) and 5 Transformer configurations with sequence length $L = 16$: `BertTiny` (2 blocks, $d = 128$, $H = 2$), `BertBase` (1 block, $d = 768$, $H = 12$), `GPT2` (1 block, $d = 768$, $H = 12$), `Llama` (1 block, $d = 768$, RoPE + SwiGLU + RMSNorm, grouped-query attention), and `ModernBert` (1 block, $d = 768$, $H = 12$, RoPE + GeGLU). Weights and inputs are random; we measure latency only, not model accuracy.

Methodology. All parties run as separate processes on one host and communicate through each framework's own local transport (Section 3.4); no network latency or bandwidth limit is emulated, and we do not report communication volume. Each primitive is run 2 times untimed (to exclude JIT compilation and cache warmup) and then timed 5 times; each Transformer block is warmed up once and timed 3 times. Parties synchronize with a barrier before every timed run, device work is synchronized before the timer stops, and we report the median. Within a platform, all reported modes of both frameworks were measured in a single session (all five modes on CPU, and four modes on GPU, where `poly` results are pending). CPU experiments use the host listed in Table 2's

Table 2: **CPU Transformer-block latency (ms) and PriX speedup** for all five approximation modes ($N = 2$ parties, $L = 16$; median of 3 timed runs). Host: AMD EPYC 7B12, 24 vCPUs; Python 3.13, JAX/jaxlib 0.12.0, PyTorch 2.14.1 (CPU). All entries were measured in one session. Lower is better.

	CrypTen Approx.		Piecewise Poly		SHAFT (Fourier+ODE)		CuRL Wavelet LUTs		Fission Evaluator	
Model	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**
BertTiny (2-Blk)	1116.92	**394.21 (2.83×)**	1214.06	**332.66 (3.65×)**	769.78	**255.53 (3.01×)**	884.35	**298.84 (2.96×)**	426.03	**97.29 (4.38×)**
BertBase (1-Blk)	2111.69	**555.14 (3.80×)**	2422.60	**512.64 (4.73×)**	1992.19	**464.97 (4.28×)**	1927.60	**453.40 (4.25×)**	2067.63	**334.52 (6.18×)**
GPT2 (1-Blk)	2725.99	**547.24 (4.98×)**	2797.50	**450.43 (6.21×)**	2483.98	**424.27 (5.85×)**	3366.24	**410.94 (8.19×)**	2690.23	**292.49 (9.20×)**
Llama (1-Blk)	2015.02	**407.47 (4.95×)**	2082.81	**390.03 (5.34×)**	2015.52	**345.54 (5.83×)**	2558.86	**385.23 (6.64×)**	2058.74	**268.46 (7.67×)**
ModernBert (1-Blk)	1520.72	**386.44 (3.94×)**	1524.23	**346.73 (4.40×)**	1536.32	**273.18 (5.62×)**	2089.26	**313.73 (6.66×)**	1378.75	**200.17 (6.89×)**

Table 3: **CPU primitive and nonlinearity latency (ms, CrypTen / PriX)** and PriX speedup (median of 5 timed runs; same host and session as Table 2).

Operation (Shape)	**CrypTen Approx.**	**Piecewise Poly**	**SHAFT (Fourier+ODE)**	**CuRL Wavelet LUTs**	**Fission Evaluator**
Private MatMul (128 × 128)	23.52 / **4.91 (4.79×)**	23.13 / **5.97 (3.87×)**	22.46 / **4.81 (4.67×)**	26.46 / **6.56 (4.03×)**	34.75 / **4.59 (7.57×)**
Private Mul (128 × 128)	4.64 / **2.63 (1.77×)**	4.64 / **1.84 (2.53×)**	5.02 / **1.89 (2.65×)**	8.62 / **4.67 (1.85×)**	6.95 / **2.77 (2.51×)**
Comparison / ReLU (32 × 64)	22.83 / **4.68 (4.88×)**	23.85 / **4.61 (5.17×)**	26.78 / **4.72 (5.67×)**	24.64 / **5.72 (4.31×)**	53.00 / **4.73 (11.21×)**
Exp (32 × 64)	15.21 / **8.14 (1.87×)**	43.27 / **11.50 (3.76×)**	14.86 / **6.33 (2.35×)**	48.23 / **14.16 (3.41×)**	12.62 / **1.11 (11.40×)**
Reciprocal (32 × 64)	80.84 / **33.62 (2.40×)**	85.78 / **29.17 (2.94×)**	73.62 / **32.70 (2.25×)**	33.38 / **6.83 (4.89×)**	21.93 / **1.05 (20.80×)**
InvSqrt (32 × 64)	46.97 / **17.91 (2.62×)**	50.21 / **22.03 (2.28×)**	53.72 / **19.71 (2.73×)**	97.08 / **15.35 (6.33×)**	22.03 / **1.26 (17.52×)**
GeLU (32 × 64)	89.70 / **40.91 (2.19×)**	43.82 / **16.38 (2.68×)**	59.12 / **13.74 (4.30×)**	21.39 / **5.90 (3.63×)**	12.54 / **1.15 (10.89×)**
SiLU (32 × 64)	71.08 / **24.27 (2.93×)**	49.93 / **15.69 (3.18×)**	63.15 / **13.41 (4.71×)**	27.30 / **5.16 (5.29×)**	11.83 / **1.20 (9.86×)**
Softmax (4 × 32 × 32)	293.28 / **103.93 (2.82×)**	333.74 / **100.37 (3.33×)**	136.18 / **55.29 (2.46×)**	284.79 / **82.01 (3.47×)**	14.64 / **1.58 (9.28×)**
LayerNorm (4 × 32 × 64)	55.72 / **22.91 (2.43×)**	54.56 / **18.13 (3.01×)**	56.83 / **17.13 (3.32×)**	50.91 / **14.32 (3.55×)**	16.36 / **2.89 (5.66×)**

caption. GPU experiments use a Google Colab instance with one NVIDIA Tesla T4. With so few repetitions, differences of roughly 10% or less between the two frameworks should be treated as within run-to-run noise.

5.2 CPU Benchmarks

Tables 2 and 3 report the CPU results. PriX has lower latency than CrypTen in all 75 CPU configurations: Transformer-block speedups range from **2.83**× to **9.20**×, and primitive speedups from 1.77× (Mul, `crypten` mode) to 20.80× (Reciprocal, `fission` mode).

- **Run-to-run noise.** ReLU has the same protocol in every mode, so its spread across modes gives a rough noise estimate. Within this session, CrypTen's ReLU took 22.8–26.8 ms in four modes and 53.0 ms in `fission` mode, where an extra evaluator process competes for the same host. PriX's ReLU took 4.6–5.7 ms. The CrypTen `fission` column is therefore likely pessimistic for CrypTen, and the very large `fission` primitive speedups (9–21×) should be read with this in mind. Across sessions on the same host, we observed differences of up to roughly 35% for identical configurations.

- **Matrix multiplication.** On CPU, CrypTen uses PyTorch's `int64` matmul, which has no BLAS backend. PriX uses FP64 GEMM on limbs (Section 3.1), which explains the 3.9–4.8× MatMul speedups outside `fission` mode. Linear layers dominate the $d = 768$ blocks, which is consistent with their larger end-to-end speedups (3.8–9.2×) compared with `BertTiny` (2.8–4.4×).

- **Comparisons and LUTs.** ReLU is 4.3–5.7× faster outside `fission` mode, consistent with fusing the local work of each SPK round. In `lut` mode, InvSqrt drops from 97.1 to 15.4 ms (6.33×) and SiLU from 27.3 to 5.2 ms (5.29×). We have not run an ablation, so we cannot say how much of these gains comes from kernel fusion and how much from the faster local transport (Section 3.4).

Table 4: **GPU (NVIDIA Tesla T4, Google Colab) Transformer-block latency (ms) and PriX speedup** ($N = 2$ parties, $L = 16$; median of 3 timed runs; all entries from one session). `poly`-mode GPU results are omitted pending a rerun with the corrected baseline described in Section 5.

	CrypTen Approx.		**SHAFT (Fourier+ODE)**		**CuRL Wavelet LUTs**		**Fission Evaluator**	
Model	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**
BertTiny (2-Blk)	1891.21	**831.48 (2.27×)**	1206.41	**497.70 (2.42×)**	1928.25	**641.74 (3.00×)**	393.31	**144.88 (2.71×)**
BertBase (1-Blk)	1651.63	**753.54 (2.19×)**	1303.92	**623.69 (2.09×)**	1680.62	**658.69 (2.55×)**	908.80	**428.51 (2.12×)**
GPT2 (1-Blk)	1648.83	**780.20 (2.11×)**	1284.63	**611.72 (2.10×)**	1611.53	**645.35 (2.50×)**	860.02	**436.71 (1.97×)**
Llama (1-Blk)	1596.48	**628.52 (2.54×)**	1284.00	**488.35 (2.63×)**	1681.80	**554.73 (3.03×)**	978.78	**354.19 (2.76×)**
ModernBert (1-Blk)	1439.01	**552.47 (2.60×)**	1094.72	**415.51 (2.63×)**	1438.62	**466.32 (3.09×)**	705.92	**239.02 (2.95×)**

Table 5: **GPU (NVIDIA Tesla T4) primitive and nonlinearity latency (ms, CrypTen / PriX)** and PriX speedup (median of 5 timed runs; same session as Table 4).

Operation (Shape)	**CrypTen Approx.**	**SHAFT (Fourier+ODE)**	**CuRL Wavelet LUTs**	**Fission Evaluator**
Private MatMul (128×128)	13.32 / **3.37 (3.96×)**	11.29 / **5.43 (2.08×)**	16.16 / **7.05 (2.29×)**	11.36 / **3.53 (3.21×)**
Private Mul (128×128)	4.06 / **4.00 (1.01×)**	4.09 / **2.60 (1.57×)**	9.77 / **5.09 (1.92×)**	4.23 / **2.74 (1.54×)**
Comparison / ReLU (32×64)	45.60 / **13.77 (3.31×)**	44.99 / **13.09 (3.44×)**	51.06 / **13.67 (3.74×)**	50.62 / **13.79 (3.67×)**
Exp (32×64)	28.31 / **16.73 (1.69×)**	27.10 / **17.57 (1.54×)**	67.27 / **24.89 (2.70×)**	6.99 / **1.52 (4.61×)**
Reciprocal (32×64)	120.16 / **65.95 (1.82×)**	134.18 / **62.83 (2.14×)**	34.20 / **12.07 (2.83×)**	8.58 / **1.59 (5.40×)**
InvSqrt (32×64)	76.28 / **37.66 (2.03×)**	81.15 / **36.33 (2.23×)**	130.45 / **12.67 (10.30×)**	7.24 / **1.74 (4.15×)**
GeLU (32×64)	142.86 / **72.66 (1.97×)**	93.29 / **25.61 (3.64×)**	24.22 / **12.61 (1.92×)**	7.33 / **1.75 (4.20×)**
SiLU (32×64)	124.01 / **46.51 (2.67×)**	92.20 / **27.90 (3.30×)**	25.96 / **12.99 (2.00×)**	7.58 / **1.60 (4.74×)**
Softmax ($4 \times 32 \times 32$)	543.45 / **221.37 (2.45×)**	225.10 / **93.67 (2.40×)**	491.56 / **190.68 (2.58×)**	7.40 / **1.59 (4.65×)**
LayerNorm ($4 \times 32 \times 64$)	92.08 / **42.26 (2.18×)**	96.84 / **45.36 (2.14×)**	149.84 / **24.11 (6.22×)**	9.34 / **3.40 (2.75×)**

- **Piecewise polynomials.** `poly` mode makes GeLU and SiLU cheaper than `crypten` mode in both frameworks (in PriX, 16.4 vs. 40.9 ms and 15.7 vs. 24.3 ms), but more expensive than CuRL's LUTs (5.9 and 5.2 ms). For exp, the piecewise polynomial is *slower* than the limit approximation (11.5 vs. 8.1 ms in PriX) because its secure comparisons cost more than 8 squarings.

5.3 GPU Benchmarks (NVIDIA Tesla T4, CUDA)

Tables 4 and 5 report the GPU results for four modes. PriX has lower latency in all 60 of these configurations, with Transformer-block speedups of **1.97×–3.09×** and primitive speedups from 1.01× (Mul, `crypten` mode, within noise) to 10.30× (InvSqrt, `lut` mode).

- **Comparisons.** CrypTen's 32×64 ReLU takes 45–51 ms on the GPU, about twice its CPU time, which is consistent with many small kernel launches per SPK round. PriX takes 13.1–13.8 ms, a 3.3–3.7× speedup.
- **Wavelet LUTs.** In `lut` mode, InvSqrt drops from 130.5 to 12.7 ms (10.30×) and LayerNorm from 149.8 to 24.1 ms (6.22×). Both frameworks use the same single-table Biorthogonal-2.2 configuration here. The gain therefore comes from executing the same protocol (truncation, one-hot rotation, table dot product) with fewer launches and a faster local transport, not from a different algorithm.
- **SHAFT.** Fourier GeLU drops from 93.3 to 25.6 ms (3.64×) and ODE Softmax from 225.1 to 93.7 ms (2.40×).
- **End-to-end.** Transformer-block speedups are smaller on GPU (1.97–3.09×) than on CPU. Plausible causes are the T4's low FP64 throughput and host-side communication, which faster GPU kernels do not reduce. We have not profiled this split.

6 Security Considerations and Limitations

PriX is a research framework operating in the **semi-honest (honest-but-curious)** model, and its current implementation has the following limitations:

1. **Correlated Randomness Providers (`TFP` vs. `TTP`):** Like CrypTen [8], PriX defaults to the Trusted First Party (`TFP`) provider, in which Party 0 generates the Beaver triples, truncation pairs, and LUT correlations. Party 0 therefore knows the dealer randomness and can recover every secret; `TFP` is suitable only for performance measurement, and all experiments in this report use it for both frameworks. Setting `cfg.mpc.provider = "TTP"` runs a separate dealer process (`TTPServer`) under the usual non-colluding third-party assumption.

2. **Pseudorandom Number Generation:** Correlated randomness is currently expanded with `SplitMix64` (inside compiled kernels) and `PCG64` (on the host), seeded from OS entropy exchanged at initialization. Neither generator is a cryptographically secure PRF, so a party that observes part of a stream may predict the rest. Replacing them with a keyed counter-mode PRF (e.g. Threefry or AES-CTR) is required before PriX can protect real data.

3. **Probabilistic Truncation Range:** PriX's TruncPr [4] is instantiated with parameter $\ell = 62$ in $\mathbb{Z}_{2^{64}}$: the opened value is $2^{64-\ell-1}(x + 2^{\ell-1} + r)$ with a uniform ℓ-bit mask r and a masked carry bit, which hides x perfectly provided the ring integer satisfies $|x| < 2^{\ell-1} = 2^{61}$. With $f = 16$, this means real values below 2^{29} in magnitude for products at precision $2f$, and below 2^{45} for values at precision f. Values outside this range give incorrect results. As usual, the truncated result may be off by one unit in the last place.

4. **Evaluation Scope:** All measurements are single-host, with no network emulation and no reported communication volume; they characterize computation and local inter-process overhead, not wide-area deployment. The two frameworks also use different local transports (Section 3.4). Model accuracy under each approximation mode is not evaluated here. PriX has been tested on CPUs and CUDA GPUs only. On TPUs, its FP64 limb matrix multiplication would rely on emulated double precision.

7 Conclusion

We presented **PriX**, a JAX/XLA framework for privacy-preserving Transformer research via Secure Multi-Party Computation. PriX implements five MPC approximation paradigms (**CrypTen**, **piecewise polynomials**, **SHAFT**, **CuRL**, and **Fission**) behind one API. It compiles the local steps of each protocol with XLA, computes only the surviving limb products in its exact FP64 ring matrix multiplication, and uses a shared-memory local transport. On a single host, it has lower latency than the PyTorch CrypTen baseline in every configuration we measured, with Transformer-block speedups of **2.83**×–**9.20**× on CPU and **1.97**×–**3.09**× on a Tesla T4 GPU. Separating the effects of kernel fusion and transport, evaluating over real networks, reporting communication volume and accuracy, and replacing the non-cryptographic PRNGs are the main next steps. PriX is open-source and available on PyPI via `pip install prix-jax`.

References

[1] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology – CRYPTO '91*, pages 420–432. Springer, 1991.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs. http://github.com/jax-ml/jax, 2018.

[3] Ye Dong, Wen jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. PUMA: Secure inference of LLaMA-7B in five minutes. *arXiv preprint arXiv:2307.12533*, 2023. https://arxiv.org/abs/2307.12533.

[4] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology – CRYPTO 2020*, volume 12171 of *Lecture Notes in Computer Science*, pages 823–852. Springer, 2020.

[5] Charles Gouert, Mehmet Ugurbil, Dimitris Mouris, Miguel de Vega, and Nektarios G. Tsoutsos. Ripple: Accelerating programmable bootstraps for FHE with wavelet approximations. In *Information Security – 27th International Conference (ISC 2024)*, volume 15155 of *Lecture Notes in Computer Science*, pages 273–293. Springer, 2024. Cryptology ePrint Archive, Paper 2024/866, https://eprint.iacr.org/2024/866.

[6] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 15718–15731, 2022.

[7] Andes Y. L. Kei and Sherman S. M. Chow. SHAFT: Secure, handy, accurate, and fast transformer inference. In *Network and Distributed System Security Symposium (NDSS)*, 2025. Cryptology ePrint Archive, Paper 2024/1965, https://eprint.iacr.org/2024/1965.

[8] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. CrypTen: Secure multi-party computation meets machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 4963–4975, 2021.

[9] Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793, 1973.

[10] Zhengyi Li, Kang Yang, Jin Tan, Wen jie Lu, Haoqi Wu, Xiao Wang, Yu Yu, Derun Zhao, Yancheng Zheng, Minyi Guo, and Jingwen Leng. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. https://arxiv.org/abs/2411.15707.

[11] Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *IEEE Symposium on Security and Privacy (S&P)*, pages 19–38, 2017.

[12] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 325–342, 2020.

[13] José Reis, Mehmet Ugurbil, Sameer Wagh, Ryan Henry, and Miguel de Vega. Wave hello to privacy: Efficient mixed-mode MPC using wavelet transforms. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2025(2):697–718, 2025. https://doi.org/10.56553/popets-2025-0083.

[14] Manuel B. Santos, Dimitris Mouris, Mehmet Ugurbil, Stanislaw Jarecki, José Reis, Shubho Sengupta, and Miguel de Vega. Curl: Private LLMs through wavelet-encoded look-up tables. In *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS)*, pages 1–31, 2024. Cryptology ePrint Archive, Paper 2024/1045, https://eprint.iacr.org/2024/1045.

[15] Mehmet Ugurbil, Dimitris Mouris, Manuel B. Santos, José Cabrero-Holgueras, Miguel de Vega, and Shubho Sengupta. Fission: Distributed privacy-preserving large language model inference. In *IEEE Conference on Communications and Network Security (CNS)*, pages 1–9, 2025. Cryptology ePrint Archive, Paper 2025/653, https://eprint.iacr.org/2025/653.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.